

	Network Administration <i>Installing and Configuring a DNS</i>	2025/2026
---	---	------------------

Objectifs : • Understand and implement DNS services • Verify and troubleshoot DNS name resolution
--

Background / Scenario

Domain Name Service (DNS) is an Internet service that maps IP addresses and fully qualified domain names to one another. In this way, DNS alleviates the need to remember IP addresses. Computers that run DNS are called *name servers*. Ubuntu ships with BIND (Berkley Internet Naming Daemon), the most common program used for maintaining a name server on Linux.

In this Lab, a dedicated DNS server needs to be deployed in a separate network namespace to decouple name resolution from address allocation. The lab uses static IPs (Lab 0) and static NAT (Lab3). It focuses entirely on DNS, keeping NAT/IP allocation simple and predictable. The objectives are:

- Understand DNS concepts: learn the role of DNS in mapping human-readable hostnames to IP addresses.
- Deploy a DNS server: set up and configure a DNS server in an isolated network namespace.
- Integrate with static IPs: test name resolution in a network with static addressing and static NAT.
- Verify DNS functionality: use client namespaces to query the DNS server and confirm correct hostname resolution.

Part 1: Verify network configuration

Step 1: Execute the Lab0's script

PCs and Gateway should already have static IPs:

- PC1 → 192.168.10.10/24
- PC2 → 192.168.20.10/24
- Gateway LAN → 192.168.10.1 / 192.168.20.1
- ISP → 209.165.200.225/27

Step 2: Test basic connectivity

#All LAN ↔ Gateway and Gateway ↔ ISP links should work.

```
sudo ip netns exec PC1 ping -c 3 192.168.10.1
sudo ip netns exec PC2 ping -c 3 192.168.20.1
sudo ip netns exec GATEWAY ping -c 3 209.165.200.225
```

Step 3: Execute Lab 2's script (SNAT+ Internet access)

```
#All LAN ↔ Gateway and Gateway ↔ ISP links should work.
sudo ip netns exec PC1 ping -c 3 192.168.10.1
sudo ip netns exec PC2 ping -c 3 192.168.20.1
sudo ip netns exec GATEWAY ping -c 3 209.165.200.225
```

Step 4: Verify the topology

Network	Purpose	
192.168.10.0/24	LAN 1 (PC1)	
192.168.20.0/24	LAN 2 (PC2)	
192.168.30.0/24	DNS Network	

Key devices, DNS is not in the same LAN, so traffic must be routed.
PC1 → 92.168.10.10
PC2 → 92.168.20.10
Gateway → Routes between all networks
DNS Server → 192.168.30.2

Part 2: Install and Configure DNS Server

Step 1: Clean Previous DNS Configuration

```
# Stops any old DNS server (named), removes old DNS namespace, deletes leftover virtual
cables (veth)
sudo pkill named 2>/dev/null || true
sudo ip netns del DNS 2>/dev/null || true
sudo ip link del veth-dns 2>/dev/null || true
sudo ip link del gw-dns 2>/dev/null || true
```

Step 2: Create the DNS Namespace

```
#Creates an isolated system called DNS, activates the loopback interface (lo)
sudo ip netns add DNS
sudo ip netns exec DNS ip link set lo up
```

Step 3: Connect DNS to the Gateway

```
# Creates a virtual Ethernet cable, one end goes to DNS, the other goes to the Gateway. This
is like plugging an Ethernet cable between two devices.
sudo ip link add veth-dns type veth peer name gw-dns
# Move interfaces, DNS namespace <----veth----> GATEWAY namespace
sudo ip link set veth-dns netns DNS
sudo ip link set gw-dns netns GATEWAY
```

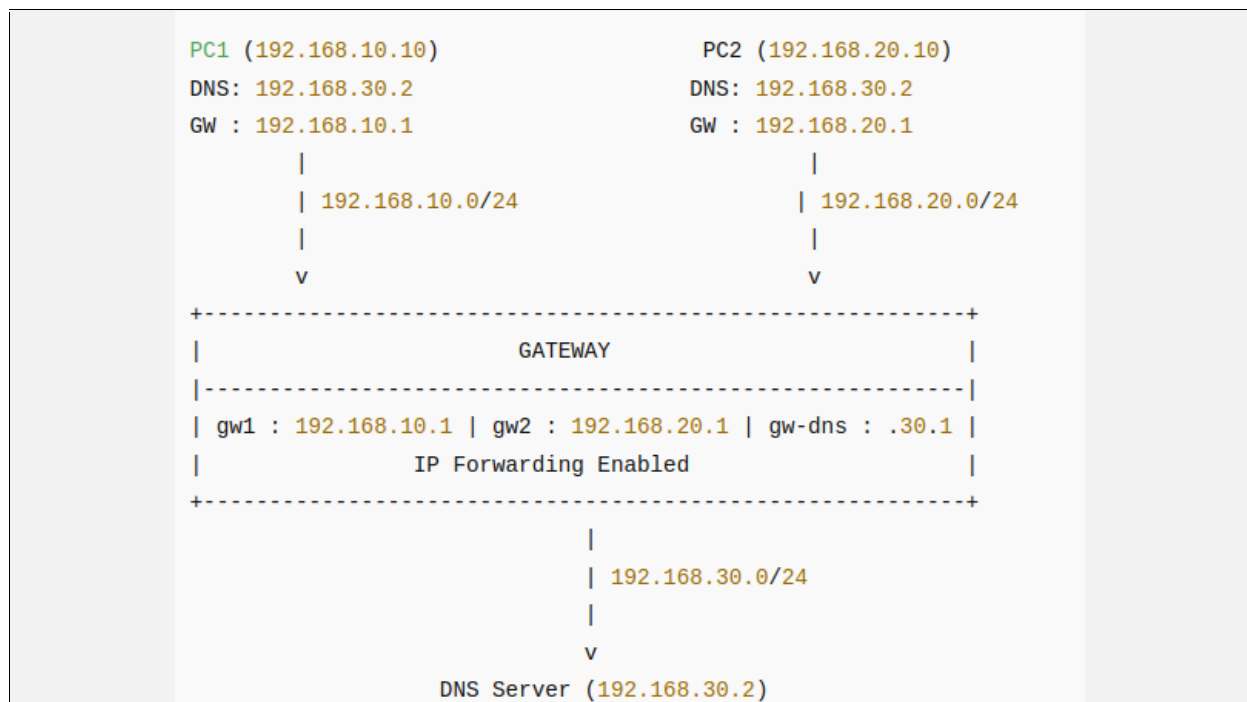
Step 4: Configure IP Addresses and Routing: DNS

```
# DNS side: assigns IP to DNS server, activates interface and sets Gateway as default route
sudo ip netns exec DNS ip addr add 192.168.30.2/24 dev veth-dns
sudo ip netns exec DNS ip link set veth-dns up
# Sends all unknown traffic to the gateway
sudo ip netns exec DNS ip route add default via 192.168.30.1
# Without a default route, DNS cannot reply to clients.
```

Step 5: Configure IP Addresses and Routing: Gateway

```
# Gateway side, Becomes router for DNS network
# IP forwarding enabled, acts as a router
sudo ip netns exec GATEWAY ip addr add 192.168.30.1/24 dev gw-dns
sudo ip netns exec GATEWAY ip link set gw-dns up
sudo ip netns exec GATEWAY sysctl -w net.ipv4.ip_forward=1
```

At this step you have the following topology diagram:



Step 6: Create DNS Zone Files (BIND)

```
# Forward lookup zone: lab.local. A DNS zone file is created for lab.local
# Creates (or overwrites) the file /etc/bind/db.lab.local, >/dev/null suppresses terminal
output. Everything between <<EOF ... EOF is written into the file
sudo tee /etc/bind/db.lab.local >/dev/null <<EOF
@ IN SOA ns.lab.local. admin.lab.local. (
    20260101 #Serial number (YYYYMMDD)
    604800   # Refresh (7 days)
    86400    # Retry (1 day)
    2419200  # Expire (28 days)
    604800 ) # Minimum TTL (7 days)
# The authoritative name server for lab.local is ns.lab.local
@ IN NS ns.lab.local. # ns.lab.local
ns IN A 192.168.30.2   # ns.lab.local → 192.168.30.2
PC1 IN A 192.168.10.10 # PC1.lab.local → 192.168.10.10
PC2 IN A 192.168.20.10 # PC1.lab.local → 192.168.20.10
EOF
# dig PC1.lab.local +short will return 192.168.10.10
# dig NS lab.local +short will return ns.lab.local.

# Reverse (DNS) lookup zone, Used by BIND to map IP to hostname
# The opposite of db.lab.local (which maps hostname → IP)
sudo tee /etc/bind/db.192.168.30 >/dev/null <<EOF
# The DNS server responsible for reverse lookups in this network is ns.lab.local
@ IN NS ns.lab.local.
2 IN PTR ns.lab.local.
EOF
```

Check the file `/etc/bind/db.lab.local`, it tells BIND everything about the domain `lab.local`. For instance, the SOA record (Start of Authority) declares this DNS server (`ns.lab.local.`) as authoritative and contains zone control parameters. Generally, this file defines the authoritative DNS zone `lab.local`, including the primary name server, administrative parameters, and host-to-IP mappings used for local name resolution. Similarly, check the file `/etc/bind/db.192.168.30`, which is used by BIND to map IP to hostname. It is indicated that the DNS server responsible for reverse lookups in this network is `ns.lab.local`. The line “2 IN PTR ns.lab.local” is the most important here, comment and explain.

Step 7: Tell BIND About the Zones

```
# Declares authoritative DNS zones
# Registers authoritative DNS zones, does not contain records, it only points to zone files
sudo tee /etc/bind/named.conf.local >/dev/null <<EOF

# Forward DNS (hostname to IP)
zone "lab.local" { # Domain name managed
    type master;    # Primary authoritative server
    file "/etc/bind/db.lab.local"; # Where records are stored
};

zone "30.168.192.in-addr.arpa" {
    type master;
    file "/etc/bind/db.192.168.30";
};
EOF
```

Step 8: Start the DNS Server

```
# Starts DNS inside the DNS namespace. This simulates a real, isolated DNS server.
sudo ip netns exec $DNS_NS pkill named 2>/dev/null || true
sudo ip netns exec DNS named -c /etc/bind/named.conf
```

Step 9: Configure DNS on Clients with dual resolvers.

```
# Tells PCs where to send DNS queries.
# Try 192.168.30.2 first
sudo ip netns exec PC1 bash -c 'echo "nameserver 192.168.30.2" > /etc/resolv.conf'
sudo ip netns exec PC1 bash -c 'echo "nameserver 8.8.8.8" >> /etc/resolv.conf'
# If no answer try 8.8.8.8
sudo ip netns exec PC2 bash -c 'echo "nameserver 192.168.30.2" > /etc/resolv.conf'
sudo ip netns exec PC2 bash -c 'echo "nameserver 8.8.8.8" >> /etc/resolv.conf'
# How to verify, inside each namespace
sudo ip netns exec PC1 cat /etc/resolv.conf
sudo ip netns exec PC2 cat /etc/resolv.conf
```

Step 10: Add Routes to the DNS Network

```
# PCs now know how to reach the DNS server.
# PC1, to reach the DNS network (192.168.30.0/24), send packets to the gateway at
192.168.10.1.
sudo ip netns exec PC1 ip route replace 192.168.30.0/24 via 192.168.10.1
sudo ip netns exec PC2 ip route replace 192.168.30.0/24 via 192.168.20.1
# How to verify
sudo ip netns exec PC1 ip route
sudo ip netns exec PC2 ip route
```

Step 11: Verify DNS Resolution

```
# If names resolve, DNS is working
sudo ip netns exec PC1 ping PC2.lab.local
sudo ip netns exec PC2 ping PC1.lab.local
sudo ip netns exec PC1 dig PC2.lab.local +short
sudo ip netns exec PC2 dig PC1.lab.local +short
```

- PC1 checks /etc/resolv.conf to see which DNS server to ask.
- Finds 192.168.30.2 (the DNS namespace).
- Sends a DNS query: “What is the IP address of PC2.lab.local?”
- DNS server replies: 192.168.20.10.

Step 12: Create a Bash script to automate the DNS lab

Create a Bash script (**enable_dns.sh**) containing the abovementioned commands to automate creating the DNS namespace, connecting it to the Gateway, assigning IPs, creating zone files, starting BIND, and configuring PCs to use DNS. Running this script allows deploying the DNS lab quickly and reliably, without manually entering each command, and ensures that the network, DNS, and routing configurations are applied correctly every time.

Step 13: Bash script to clean and reset the DNS lab

Create a Bash script (**clean_dns.sh**) to reset and clean the DNS lab environment. The script should automatically perform the following tasks: stop any running DNS server processes (named) inside the DNS namespace, delete the DNS namespace and remove all associated virtual Ethernet interfaces (veth-dns and gw-dns), remove all BIND zone files and configuration files. This will ensure that after running the script, the DNS lab environment can be safely re-run from scratch, without leftover files, processes, or network interfaces causing errors. It helps quickly resetting the lab, supporting repeated experiments or testing different configurations, while keeping the environment clean and consistent.

Part 2: Configure DNS Server resolve external hostnames

So far, DNS only resolves your lab hostnames (PC1.lab.local, PC2.lab.local). To extend the DNS lab so that the client namespaces can resolve external hostnames (like google.com) or reach external IPs (8.8.8.8), we need to add real Internet access, which is different from the current offline DNS lab. So, make sure your Gateway namespace has an interface connected to an external network (or simulated ISP namespace).

Step 1: Ensure Gateway has NAT and forwarding

```
# Execute static Nat script, so all traffic leaving PC1 or PC2 will appear with a public IP on the ISP side.
./enabling-static-nat.sh
# Check connectivity
sudo ip netns exec PC1 ping -c 3 209.165.200.225
```

Step 2: Configure DNS forwarding on your lab DNS server

```
# Edit /etc/bind/named.conf.options to allow forwarding for external domains.
sudo ip netns exec DNS vi /etc/bind/named.conf.options

# recursion yes; allows your DNS server to query external DNS servers.
# forwarders, tells BIND where to ask if a domain isn't local.
options {
    directory "/var/cache/bind"; # Where BIND stores cache and temporary files
    recursion yes;               # Allows BIND to Query other DNS servers
    allow-query { any; };        # Accept DNS queries from any IP, OK for a lab!!
    listen-on { 192.168.30.2; };
    forwarders { 8.8.8.8; 8.8.4.4; }; # If a name is not in local zones ask Google public DNS
};
# Reload BIND:
sudo ip netns exec DNS named -c /etc/bind/named.conf
```

Step 3: Test External Connectivity

```
# Test of DNS and static NAT is correctly forwarding both DNS queries and external traffic.
sudo ip netns exec PC1 ping 8.8.8.8 # tests raw IP connectivity
sudo ip netns exec PC1 dig google.com # tests DNS resolution through your lab DNS
sudo ip netns exec PC2 ping 8.8.8.8
sudo ip netns exec PC2 dig google.com
# For instance, ping google.com
PC1 → 192.168.30.2, BIND checks local zones, Not found, Forwards to 8.8.8.8, Gets reply,
Caches it, Responds to PC1,
```

Step 5: Update scripts

Update your scripts to reflect the abovementioned changes, allowing the lab to be cleaned and re-executed while testing the DNS lab with internal and external Internet name resolution, and then perform the following tests.

Lab 4 – Installing and configuring a DNS

```
sudo ip netns exec PC1 ping -c 1 192.168.30.2
sudo ip netns exec PC2 ping -c 1 192.168.30.2
sudo ip netns exec PC1 dig PC2.lab.local +short
sudo ip netns exec PC1 dig -x 192.168.30.2 +short
sudo ip netns exec PC1 dig google.com +short
# You can eventually debug as follows. Silent success/OK means configuration is valid
sudo ip netns exec DNS named-checkconf
sudo ip netns exec DNS named-checkzone lab.local /etc/bind/db.lab.local
```


Deliverables (To be uploaded on the Platform)

In this lab, you have designed and configured a Linux-based DNS environment using network namespaces. You have deployed a DNS server in a separate namespace and connect it to the Gateway namespace. Two LAN hosts (PC1 and PC2), as in Lab 0, have been configured to use the DNS server for hostname resolution. The main report document (Word/PDF, 10 pages max) must include the following:

- **DNS Configuration**
 - Namespace and network setup: IP addresses of DNS server and Gateway, routing tables showing client-to-DNS paths.
 - Explanation: how DNS resolves hostnames to IPs and how network namespaces emulate isolated hosts and servers.
- **Verification and Testing**
 - DNS functionality: ping and dig tests from PC1 and PC2 using hostnames.
 - Routing verification: check routes in each namespace with ip route.
 - Traffic verification: use tcpdump to capture DNS queries and replies between clients and the DNS server.
- **Conclusion**
 - DNS server deployed successfully, hostname resolution working for both LAN hosts. Explain the role of DNS in network communication and benefits of using Linux network namespaces for lab emulation.

A single compressed file named in the format “lastname.firstname-Lab4.zip”, must be uploaded on the platform (see deadlines) and must include:

- The main report document (Word or PDF)
- All developed Bash scripts